

Python Assignment: "You're a wizard Harry"

Part 1: The Games and the Headmaster

Question 1: The Wizard's Chess Solver

The autumn term has just begun at Hogwarts School of Witchcraft and Wizardry. While relaxing by the fire in the Gryffindor common room, you watch pieces brutally smash each other across a checkered board. Ron Weasley confidently declares, "*That's wizard's chess. I see you've worked that out.*" Intrigued by the strategic depth, you decide to prove that even magical games can be mathematically quantified. Your first task is to build an AI to perfectly solve a 3×3 grid variant of Wizard's Chess.

- **The Arena:** A 3×3 enchanted chessboard representing 9 squares.
- **The Actors:** Two competing houses, Gryffindor (plays first) and Slytherin.
- **The Rules:** Players take turns claiming one empty square at a time. The first house to claim three squares in a horizontal, vertical, or diagonal line achieves checkmate and wins. If all squares are claimed without a checkmate, the game is a stalemate (a draw).

Because this is a finite, perfect-information game, you realize you can solve it completely using the Minimax algorithm. The algorithm assumes both players play optimally, assigning a utility to terminal game states (e.g., +1 if Gryffindor wins, -1 if Slytherin wins, and 0 for a draw). Gryffindor acts as the *Maximizer*, choosing the move that leads to the highest possible value, while Slytherin acts as the *Minimizer*.

Task: Implement an OOP architecture to find the subgame perfect equilibrium strategy for both houses using backward induction—recursively exploring the game tree down to the terminal states, assigning utilities, and propagating those values back up.

1. Implement the `ChessState` class. It must maintain the sequence of historical claims and the current 3×3 grid state.
2. Implement a method `is_terminal()` that returns True if a checkmate is achieved or the grid is full.
3. Implement a function `backward_induction(state)` that computes the minimax value of the state.

4. The function must populate two global dictionaries mapping the stringified history (e.g., "048") to a probability distribution over the optimal valid moves (e.g., {"2": 1.0, "3": 0.0}). Serialize and save these dictionaries as `policy_gryffindor.json` and `policy_slytherin.json`.

Scoring Scheme (25 Marks Total):

- **5 Marks:** Correctly identifies terminal states (`is_terminal` and `is_checkmate` logic).
- **5 Marks:** Accurately computes utility values for win, loss, and draw scenarios.
- **5 Marks:** Recursively explores all valid moves without exceeding recursion limits.
- **10 Marks:** Output JSON dictionaries perfectly match the expected policy mapping for all reachable histories.

Question 2: The Marauder's Simulation Engine

Word of your unbeatable chess AI quickly reaches the Headmaster's office. Professor Dumbledore is highly intrigued and wishes to host a school-wide exhibition match where students can test their wits against your creation. However, the portraits and ghosts cannot read JSON files. He tasks you with building a Marauder's Simulation Engine—an interactive Command-Line Interface (CLI). You think to yourself, *"I solemnly swear that I am up to no good,"* as you begin coding the interface to unleash the optimal Slytherin AI against the student body.

Task: Using the JSON policies generated in Question 1, build the CLI engine.

- Create a `GameEngine` class that loads `policy_gryffindor.json` in its constructor.
- Implement a `play_match()` method. The human plays as Slytherin.
- The engine must print the grid to the console, prompt the user for an integer input (0-8), validate the move, and update the state.
- If the human makes a move, the engine must look up the new history string in the loaded JSON policy, execute the AI's deterministic optimal move, and print the updated grid.
- Handle `KeyError` exceptions gracefully if a history state is somehow not found in the JSON file, defaulting to a random valid move.

Scoring Scheme (15 Marks Total):

- **5 Marks:** Properly initializes the engine and loads the JSON policy file (throws appropriate custom exception if missing).
- **5 Marks:** Correctly tracks and updates the game history and board state after a valid user input.
- **5 Marks:** Gracefully handles `KeyError` exceptions for missing histories by falling back to a valid random move.

Part 2: The Ministry's Dilemma

Question 3: The Floo Powder Allocator

Time Limit: 1.0s — Memory Limit: 256MB

Following the success of the exhibition match, the Ministry of Magic approaches you with a dire logistical problem. Hermione Granger unrolls a massive blueprint of the Floo Network across your desk. *"It's not magic, it's logic—a puzzle,"* she explains. When transporting a magical artifact through the network, the fireplaces only allow Floo Powder requests in specific, fixed "scoop" sizes (measured in Ounces). The Ministry is wasting powder and needs an algorithmic allocator.

You are given an array of K allowed scoop sizes and a target artifact weight of T Ounces. You need to find the **minimum number of scoops** you must use to transport exactly T Ounces. You can use the same scoop size an infinite number of times. If it is impossible to form exactly T Ounces using the given scoop sizes, output -1 .

Input Format:

- The first line contains two integers K ($1 \leq K \leq 100$) and T ($1 \leq T \leq 10^4$).
- The second line contains K space-separated integers, representing the allowed scoop sizes ($1 \leq \text{size}_i \leq 1000$).

Output Format: Print a single integer: the minimum number of scoops required, or -1 if impossible.

Sample Input 1:

```
3 11
1 2 5
```

Sample Output 1:

```
3
```

Explanation: The optimal scoops are $5 + 5 + 1 = 11$, which requires 3 scoops.

Sample Input 2:

```
2 3
2 4
```

Sample Output 2:

```
-1
```

Scoring Scheme (20 Marks Total):

- **5 Marks:** Passes basic test cases (small T , typical scoop combinations).
- **10 Marks:** Passes hidden large test cases within the 1.0s time limit (requires $O(T \times K)$, brute force will TLE).
- **5 Marks:** Correctly identifies impossible configurations and outputs -1 .

Question 4: The Parseltongue Log Translator

Time Limit: 1.0s — Memory Limit: 256MB

Just as you finalize the Floo Powder algorithms, an emergency is declared in the Department of Mysteries. An ancient artifact has begun spewing critical error logs in various unknown languages, including Ancient Runes and Parseltongue. Albus Dumbledore hands you a stack of crumbling translation texts, reminding you that “*Words are, in my not-so-humble opinion, our most inexhaustible source of magic.*”

You must translate these logs into Standard English. Just as playing the right notes on an enchanted flute is the only way to bypass a three-headed dog, executing the precise shortest sequence of translations is the only way to debug this artifact before it overloads.

You have a master dictionary of E valid English words. You also have an aggregation of D legacy spellbooks. Each spellbook entry provides a one-way, single-step translation from a word U in one language to a word V in another language.

Given an error log consisting of W words, translate each word into a valid English word using the **fewest number of translation steps** possible.

- If a word can reach multiple English words in the same minimum number of steps, choose the lexicographically smallest English word to break the tie.
- If a word cannot be translated into any valid English word through any path, replace it with the exact string [ERROR].

Input Format:

- The first line contains three integers E , D , and W ($1 \leq E, D, W \leq 10^5$).
- The second line contains E space-separated strings, representing the valid English vocabulary.
- The next D lines contain two space-separated strings U and V , indicating that word U can be directly translated to word V .
- The final line contains W space-separated strings representing the original error log.

Output Format: Print a single line containing W space-separated strings: the translated English log.

Sample Input 1:

```
3 5 4
file not found
fichier archivo
archivo file
pas no
no not
trouve found
fichier pas trouve random
```

Sample Output 1:

```
file not found [ERROR]
```

Scoring Scheme (25 Marks Total):

- **5 Marks:** Passes simple direct translation test cases.
- **10 Marks:** Correctly identifies the shortest path for translations within the time limit
- **5 Marks:** Successfully resolves shortest-path ties by selecting the lexicographically smallest English word.
- **5 Marks:** Correctly outputs [ERROR] when no English word can be reached.

Part 3: The Dark Arts of Scripting

Question 5: The Horcrux Sweeper

"Seven! Is it not the most powerfully magical number?" — Tom Riddle

The ultimate battle approaches. Harry, Ron, and Hermione have discovered that Voldemort has digitized fragments of his soul (Horcruxes) and scattered them across a deeply nested, labyrinthine file directory structure simulating the British Isles.

A file is definitively a Horcrux if, and only if, it meets **both** of the following conditions:

1. Its file extension is exactly **.hx**
2. Its file size on disk is exactly **7 bytes**.

Task: Write a Python script utilizing the `os` module. Define a function `purge_horcruxes(root_path)` that takes a string representing the absolute path to the root directory of the search.

The function must:

- Recursively traverse the directory tree starting from `root_path`.
- Identify every valid Horcrux based on the two conditions above.
- For every identified Horcrux, neutralize it by changing its file extension from `.hx` to `.destroyed` using the `os.rename` function.
- Return a Python `list` containing the **relative paths** (relative to `root_path`) of all the files you successfully neutralized. The returned list must be sorted alphabetically.

Scoring Scheme (15 Marks Total):

- **4 Marks:** Accurately recursively searches the directory and targets files with the `.hx` extension (ignoring similar extensions like `.hx1`).
- **4 Marks:** Correctly verifies the file size is exactly 7 bytes using `os.path.getsize`.
- **4 Marks:** Correctly changes the file extension to `.destroyed` using `os.rename`.
- **3 Marks:** Returns the correct list of relative paths strictly sorted in alphabetical order.

Solutions

Solution 1: Wizard's Chess Solver

```
1 import json
2 import math
3
4 policy_gryffindor = {}
5 policy_slytherin = {}
6
7 class ChessState:
8     def __init__(self, history=None):
9         self.history = history if history else []
10        self.grid = ['0'] * 9
11        for i, move in enumerate(self.history):
12            self.grid[move] = 'G' if i % 2 == 0 else 'S'
13        self.actor = 'G' if len(self.history) % 2 == 0 else 'S'
14
15    def get_valid_moves(self):
16        return [i for i, val in enumerate(self.grid) if val == '0']
17
18    def is_checkmate(self):
19        lines = [(0,1,2), (3,4,5), (6,7,8), (0,3,6), (1,4,7), (2,5,8),
20        (0,4,8), (2,4,6)]
21        for a, b, c in lines:
22            if self.grid[a] == self.grid[b] == self.grid[c] != '0':
23                return True
24        return False
25
26    def is_terminal(self):
27        return self.is_checkmate() or len(self.history) == 9
28
29    def get_utility(self):
30        if self.is_checkmate():
31            return 1.0 if (len(self.history) - 1) % 2 == 0 else -1.0
32        return 0.0
33
34    def backward_induction(state):
35        if state.is_terminal():
36            return state.get_utility()
37
38        best_val = -math.inf if state.actor == 'G' else math.inf
39        best_move = -1
40
41        for move in state.get_valid_moves():
42            next_state = ChessState(state.history + [move])
43            val = backward_induction(next_state)
44
45            if state.actor == 'G':
46                if val > best_val: best_val, best_move = val, move
47            else:
48                if val < best_val: best_val, best_move = val, move
49
50        strategy = {str(i): 1.0 if i == best_move else 0.0 for i in range(9)}
51        hist_str = "".join(map(str, state.history))
```

```

52     if state.actor == 'G': policy_gryffindor[hist_str] = strategy
53     else: policy_slytherin[hist_str] = strategy
54
55     return best_val
56
57 # Execution
58 backward_induction(ChessState())
59 with open('policy_gryffindor.json', 'w') as f: json.dump(
    policy_gryffindor, f)

```

Solution 2: Marauder's Simulation Engine

```

1 import json
2 import random
3
4 class GameEngine:
5     def __init__(self, policy_path):
6         try:
7             with open(policy_path, 'r') as f:
8                 self.policy = json.load(f)
9         except FileNotFoundError:
10            raise Exception("Policy JSON not found. Run solver first.")
11        self.history = []
12        self.grid = ['-'] * 9
13
14    def display(self):
15        for i in range(0, 9, 3):
16            print(f"{self.grid[i]} | {self.grid[i+1]} | {self.grid[i+2]}")
17        print("-----")
18
19    def play_match(self):
20        print("You are Slytherin. Gryffindor (AI) moves first.")
21        while len(self.history) < 9:
22            hist_str = "".join(map(str, self.history))
23
24            # AI Turn (Gryffindor)
25            if len(self.history) % 2 == 0:
26                try:
27                    strat = self.policy[hist_str]
28                    move = int(max(strat, key=strat.get))
29                except KeyError:
30                    valid = [i for i, x in enumerate(self.grid) if x ==
31                        '-']
32                    move = random.choice(valid)
33                    self.grid[move] = 'G'
34                    self.history.append(move)
35                    print(f"AI chose square {move}")
36                    self.display()
37
38            # Human Turn (Slytherin)
39            else:
40                move = int(input("Enter your move (0-8): "))
41                if self.grid[move] != '-':
42                    print("Invalid move. Try again.")
43                    continue
44                self.grid[move] = 'S'

```

```

44         self.history.append(move)
45         self.display()

```

Solution 3: The Floo Powder Allocator (DP)

This is a variation of the classic Coin Change problem, solved in $O(T \times K)$ time.

```

1 def solve_downloader():
2     import sys
3     input_data = sys.stdin.read().split()
4     if not input_data:
5         return
6
7     K = int(input_data[0])
8     T = int(input_data[1])
9     chunks = [int(x) for x in input_data[2:2+K]]
10
11     dp = [float('inf')] * (T + 1)
12     dp[0] = 0
13
14     for amount in range(1, T + 1):
15         for chunk in chunks:
16             if amount - chunk >= 0:
17                 dp[amount] = min(dp[amount], dp[amount - chunk] + 1)
18
19     print(dp[T] if dp[T] != float('inf') else -1)
20
21 # solve_downloader()

```

Solution 4: The Parseltongue Translator (BFS)

This solution uses a BFS starting from all target English words and traversing the graph backward. This solves the problem efficiently in $O(V + E)$ time.

```

1 def solve_translator():
2     import sys
3     from collections import defaultdict, deque
4
5     input_data = sys.stdin.read().split()
6     if not input_data:
7         return
8
9     E = int(input_data[0])
10    D = int(input_data[1])
11    W = int(input_data[2])
12
13    english_words = set(input_data[3:3+E])
14    idx = 3 + E
15
16    # Store edges backwards: V -> U
17    reverse_adj = defaultdict(list)
18    for _ in range(D):
19        u, v = input_data[idx], input_data[idx+1]
20        reverse_adj[v].append(u)
21        idx += 2
22

```



```

23     log = input_data[idx:]
24
25     q = deque()
26     dist = {}
27     best_eng = {}
28
29     # Initialize Multi-Source BFS
30     for ew in english_words:
31         q.append(ew)
32         dist[ew] = 0
33         best_eng[ew] = ew
34
35     while q:
36         curr = q.popleft()
37         d = dist[curr]
38         eng = best_eng[curr]
39
40         for prev in reverse_adj[curr]:
41             if prev not in dist:
42                 dist[prev] = d + 1
43                 best_eng[prev] = eng
44                 q.append(prev)
45             elif dist[prev] == d + 1:
46                 # Tie-breaker: Lexicographically smaller English word
47                 if eng < best_eng[prev]:
48                     best_eng[prev] = eng
49
50     result = []
51     for word in log:
52         if word in best_eng:
53             result.append(best_eng[word])
54         else:
55             result.append("[ERROR]")
56
57     print(" ".join(result))
58
59 # solve_translator()

```

Solution 5: The Horcrux Sweeper (OS Module)

This solution strictly utilizes the `os` module to traverse, validate, rename, and path-format files dynamically.

```

1  import os
2
3  def purge_horcruxes(root_path):
4      neutralized_files = []
5
6      # Traverse the directory structure top-down
7      for dirpath, dirnames, filenames in os.walk(root_path):
8          for file in filenames:
9
10             # Check Condition 1: File extension
11             if file.endswith('.hx'):
12                 full_path = os.path.join(dirpath, file)
13
14             # Check Condition 2: File size is exactly 7 bytes

```

```

15         if os.path.getsize(full_path) == 7:
16
17             # Split the extension and prepare the new name
18             base_name, _ = os.path.splitext(full_path)
19             new_full_path = base_name + '.destroyed'
20
21             # Neutralize the Horcrux
22             os.rename(full_path, new_full_path)
23
24             # Calculate the relative path for reporting
25             rel_path = os.path.relpath(new_full_path, root_path
)
26
27             neutralized_files.append(rel_path)
28
29     # Return alphabetically sorted list of neutralized files
30     return sorted(neutralized_files)

```